

AKLO Test Plan

Team 12: Automated Kinetic Linking Optimization

Purpose

The purpose of these tests is to validate success in translating EMG data into motorized Morse code output. Specifically, this objective will be achieved through 3 separate tests addressing each aspect of the AKLO system: the translation of EMG data into dots and dashes, the translation of dots and dashes into English through Morse code, and accurate motor vibration in accordance with the interpreted Morse code. Together, these tests review all features of the team's designed prototype.

Scope

The presented AKLO test plan is limited in that it does not verify the performance of the system's input board. Instead, simulated EMG data is used as input and assumed to be representative of the data provided by the input board. Otherwise, the test plan comprehensively reviews functionality in all components beyond the input board (i.e., in the processing board and output board) to ensure accurate motor vibrations, software functionality, and device usability. **Table 1** outlines each of these tests and the corresponding requirements they demonstrate in a cross-reference matrix.

Table 1: Requirement/Verification Cross-Reference Matrix

Classes of Verification: I- First Article, II-Environmental, III-Acceptance Test, IV- None

Methods of Verification: A- Analysis, T-Test, D-Demonstration, I- Inspection

Project Requirements	Test Names	Verification Class				Location of Test Results
		I	II	III	IV	
	1. Software					
Accurate translation	1.1 Software correctly translates gestures from raw EMG data to dots and dashes	A				Appendix B
Accurate translation	1.2 Software correctly translates dots and	A				Appendix C

	dashes into English using Morse code					
	2. Hardware					
Accurate translation	2.1 Motors correctly render input dot and dash pattern as vibrations	D				Appendix D

Testing Procedure

The following section reviews the purpose, methodology, procedure, and pass/fail criteria of each test conducted by the team.

1.1 Dot-Dash Detection from Raw EMG Data

The purpose of this test is to verify that the software algorithm can correctly process raw EMG signals and classify detected gestures into Morse code symbols (dot or dash) based on signal threshold detection and gesture duration. The threshold dictates when EMG signals are strong enough to imply purposeful muscle contraction. The duration of that recorded contraction is then used to decide whether a dot or dash is transmitted.

Due to an unexpected hardware failure with the provided input board, the testing methodology was adapted to utilize a public online dataset. The simulated raw EMG test data files used by the team were generated by expanding on the dataset's 8-channel format with custom Morse code sequences. Then, mathematical variance and randomness were added across all channels to simulate natural signal noise and validate the algorithm under real-world conditions.

The phrase "how are you" was selected as the primary test input because it challenges the system to detect all possible types of input within a single sequence: dot-only characters (h, e), dash-only characters (o), and mixed dot-dash characters (w, a, r, y, u). Moreover, this sequence also requires the system to correctly identify between characters grouped into words (e.g., h, o, and w should be recognized as a group for the word "how") and groups of characters separated by spaces (e.g., "how" and "are" should be separated by a space). Altogether, these challenges comprehensively review the prototype's detection capabilities.

Two versions of the "how are you" test file were created. The first file tests a clean signal with no noise, serving as the baseline to verify correct detection under ideal conditions. The second introduces realistic noise across all channels to validate the system's robustness under real-world

conditions. The modelled noise includes three different types commonly observed in surface EMG recordings: brief muscle twitches during rest periods (short, above-threshold spikes that occur during gaps between gestures), signal dropouts during muscle contraction (brief loss of electrode contact or muscle relaxation, causing the signal to dip to near-zero mid-gesture), and abnormally strong signal bursts during muscle contraction (sudden high-amplitude spikes).

These tests are passed if and only if the decoded outputs for both the clean and noisy "how are you" test files match their expected outputs (i.e., the array [. . . . | - - - | . - - || . - | . - . | . || - . - - | - - - | . . - |]) with 100% accuracy.

Procedure:

1. Place the two test files, *test_how_are_you_morse.csv* and *test_how_are_you_morse_with_noise.csv*, into the same project directory as the main software pipeline, *threshold_dots_dashes.py*.
2. Open *threshold_dots_dashes.py* in a preferred text editor or IDE.
3. Locate the filename variable in the main function, change it to "test_how_are_you_morse.csv", and save the file.
4. Execute the software and record the resulting output array sequence printed to the console.
5. Compare the software-generated array with the expected Morse code sequence [. . . . | - - - | . - - || . - | . - . | . || - . - - | - - - | . . -]. Determine if the output matches the expected sequence.
6. Return to the text editor or IDE, update the filename variable to "test_how_are_you_morse_with_noise.csv", and save the file.
7. Execute the software and record the resulting output array sequence printed to the console.
8. Compare the output array with the same expected Morse code sequence from step 5. Determine if the output achieves 100% accuracy.

1.2 Translation of Dots and Dashes into Alphanumeric Characters using Morse Code

The purpose of this test is to verify that the developed AKLO software correctly uses Morse code to translate dot and dash sequences into their corresponding alphanumeric characters.

To test the software's Morse code translation capabilities, five arrays of dots and dashes with known alphanumeric translations were hardcoded and fed through the developed Morse code decoding function. Together, the five arrays challenge the function to translate all possible cases of input: a single character, a single word, a numeric string, a multi-word sentence, and a word with an invalid character.

The overall accuracy of each test is provided in the terminal as output when following the procedure described in Appendix D. The software calculates overall accuracy by first determining the accuracy of each interval of vibration/rest in a test case using the following equation:

$$\text{Interval Accuracy} = 1 - \frac{|\text{Measured Duration} - \text{Expected Duration}|}{\text{Expected Duration}}$$

The overall test accuracy is then calculated as the average of all interval accuracies in the tested phrase. This method is used so that small timing shifts potentially introduced by motor response delay or software control latency do not affect test results and each vibration or spacing segment can be evaluated independently.

To measure the motor vibration output by the system, the audio of the vibration motors is recorded to generate a corresponding visual waveform. This audio waveform is then compared against the expected time duration plot, which illustrates exactly when the motors should vibrate. This visual comparison precisely confirms that the duration of each physical dot and dash perfectly matches the software's expected timing.

Appendix B

Table B.1 Dot-Dash Detection from Raw EMG Data, “*test_how_are_you_morse.csv*”

Line of Output	Expected Value	Actual Value
1	DOT	DOT
2	DOT	DOT
3	DOT	DOT
4	DOT	DOT
5	LETTER COMPLETE:	LETTER COMPLETE:
6	DASH	DASH
7	DASH	DASH
8	DASH	DASH
9	LETTER COMPLETE: ---	LETTER COMPLETE: ---
10	DOT	DOT
11	DASH	DASH
12	DASH	DASH
13	LETTER COMPLETE: .--	LETTER COMPLETE: .--
14	WORD BOUNDARY	WORD BOUNDARY
15	DOT	DOT
16	DASH	DASH
17	LETTER COMPLETE: .-	LETTER COMPLETE: .-
18	DOT	DOT
19	DASH	DASH
20	DOT	DOT
21	LETTER COMPLETE: .-.	LETTER COMPLETE: .-.
22	DOT	DOT
23	LETTER COMPLETE: .	LETTER COMPLETE: .
24	WORD BOUNDARY	WORD BOUNDARY
25	DASH	DASH
26	DOT	DOT
27	DASH	DASH
28	DASH	DASH
29	LETTER COMPLETE: -..	LETTER COMPLETE: -..
30	DASH	DASH
31	DASH	DASH
32	DASH	DASH
33	LETTER COMPLETE: ---	LETTER COMPLETE: ---
34	DOT	DOT
35	DOT	DOT
36	DASH	DASH
37	LETTER COMPLETE: ...-	LETTER COMPLETE: ...-

Total accuracy: 100%, Test performed by: Andrew Hace

Table B.2 Dot-Dash Detection from Raw EMG Data, “*test_how_are_you_morse_with_noise.csv*”

Line of Output	Expected Value	Actual Value
1	DOT	DOT
2	DOT	DOT
3	DOT	DOT
4	DOT	DOT
5	LETTER COMPLETE:	LETTER COMPLETE:
6	DASH	DASH
7	DASH	DASH
8	DASH	DASH
9	LETTER COMPLETE: ---	LETTER COMPLETE: ---
10	DOT	DOT
11	DASH	DASH
12	DASH	DASH
13	NOISE (too short)	NOISE (too short)
14	LETTER COMPLETE: .--	LETTER COMPLETE: .--
15	WORD BOUNDARY	WORD BOUNDARY
16	DOT	DOT
17	DASH	DASH
18	LETTER COMPLETE: .-	LETTER COMPLETE: .-
19	DOT	DOT
20	DASH	DASH
21	DOT	DOT
22	LETTER COMPLETE: .-.	LETTER COMPLETE: .-.
23	DOT	DOT
24	NOISE (too short)	NOISE (too short)
25	LETTER COMPLETE: .	LETTER COMPLETE: .
26	WORD BOUNDARY	WORD BOUNDARY
27	DASH	DASH
28	DOT	DOT
29	DASH	DASH
30	DASH	DASH
31	LETTER COMPLETE: -..	LETTER COMPLETE: -..
32	DASH	DASH
33	DASH	DASH
34	DASH	DASH
35	LETTER COMPLETE: ---	LETTER COMPLETE: ---
36	DOT	DOT
37	DOT	DOT
38	DASH	DASH
39	LETTER COMPLETE: ..-	LETTER COMPLETE: ..-

Total accuracy: 100%, Test performed by: Becker Cheng

Appendix C

Table C.1 Dots and Dashes into Alphanumeric Characters Translation

Test Case	Input Array Sequence	Expected Alphanumeric Character Output	Actual Alphanumeric Character Output
1 (Single Character)	[. -]	“a”	“a”
2 (Single Word)	[.]	“hi”	“hi”
3 (Multi-Word Sentence)	[. . . . - - - . - - . - . - . . - . - - - - - . . -]	“how are you”	“how are you”
4 (Numeric String)	[. - - - - . . - - - - - ]	“12345”	“12345”
5 (word with invalid character)	[. - . - . - - . . . - . - .]	“?bc”	“?bc”

Total accuracy: 100%, Test performed by: Sylvia Hallak

Appendix D

Hardware Functionality – Accurate Motor Vibration	
Method of Verification: Demonstration	
Type of Test: First Article	
Date(s)/ Total Times: 3/26/2026	Location: Capstone Lab, DL 761
Test Equipment: Raspberry Pi processing board, haptic output board, haptic motors, smartphone for audio recording, laptop or Raspberry Pi for running the analysis code	
Test Engineer: Junyi Cheng Witness: Becker Cheng	
☒ Passes – The recorded overall timing of tested motor vibration outputs is at least 90% accurate when compared against the expected timing of motor behavior. ☐ Fails – The recorded overall timing of tested motor vibration outputs is less than 90% accurate when compared against the expected timing of motor behavior.	
Test Procedure: 1. Place the files <i>MainCode.py</i>, <i>test_hi_morse_with_noise.csv</i>, <i>test_how_are_you_morse.csv</i>, and <i>hardware_validate.py</i> in the same project directory on the Raspberry Pi. 2. In a quiet environment, position an audio-recording device (e.g., a smartphone or laptop) approximately 2–5 cm away from the haptic motors connected to the Raspberry Pi. 3. On the Raspberry Pi, open <i>MainCode.py</i>, locate the filename variable in the main() function, change it to "test_hi_morse_with_noise.csv", and save the file. 4. Start recording audio and then execute the script on the Raspberry Pi using python <i>MainCode.py</i>, allowing the motor to complete its full vibration sequence. 5. Stop the recording after the sequence finishes, save the audio as <i>hi_audio.mp3</i>, and transfer or place the file into the same project directory on the Raspberry Pi. 6. Open the recorded audio file on a smartphone or computer and view the recorded waveform. Through visual inspection, identify the start and end times of each interval of vibration/rest.	

7. Record start and end timestamps of each identified interval in seconds. Label each interval as either a dot, dash, inter-letter rest, or inter-word space.
8. Open *hardware_validate.py* and update the test interval list (i.e., `hi_test_intervals` or `how_are_you_test_intervals`) by replacing the start time, end time, and label of each interval with the recorded values from step 7.
9. Run *hardware_validate.py*. The script will automatically generate plots comparing the recorded test signals with the theoretical signals for each case.
10. Review the generated plots (saved as image files). Then, check the terminal output for interval-by-interval accuracy and overall accuracy.
11. Record overall accuracy. Determine whether the overall interval timing accuracy is at least 90%. If not, mark the test as a failure.
12. Repeat Steps 3 through 11 with the filename variable changed to `"test_how_are_you_morse.csv"`, and update the corresponding test intervals in the code using the new audio recording.

Test Results:

Test Case	Expected Accuracy	Real Accuracy
1 'Hi'	100%	91.73%
2 'How are you'	100%	91.20%

Action Items:

None

Test Engineer: Junyi Cheng, 03/26/2026	Witness: Becker Cheng, 03/26/2026
---	--------------------------------------

Appendix E: Test Status Master List

TEST #	TYPE OF TEST	TEST CATEGORY	TEST NAME	OWNER	PLANNED DATE	ACTUAL DATE	STATUS	ACTION ITEM
1	First Article	Software	Dot-Dash Detection from Raw EMG Data	Becker/Drew	3/12/2026	3/12/2026	Passed	N/A, test passed
2	First Article	Software	Translation of Dots and Dashes into Alphanumeric Characters using Morse Code	Sylvia	3/24/2026	3/24/2026	Passed	N/A, test passed
3	First Article	Hardware	Accurate Motor Vibration	Jacob	3/26/2026	3/26/2026	Passed	N/A, test passed